



OBJECT ORIENTED PROGRAMMING (23EN1202)

UNIT-2: INTRODUCTION TO OOP AND PYTHON

STRINGS

UNIT – II

5 Hours

FUNCTIONS AND MODULES:

Need for functions, Function definition, Function call, Scope, Return statement, Lambda functions, Recursive functions, Modules.

(Chapter 5: 5.1 to 5.11)

PYTHON STRINGS:

String operations, Immutable, string formatting operator, built-in string methods, string slices, membership operator, comparing strings, Iterating strings.

(Chapter 6: 6.1 to 6.9)

Unit-2

Python Strings

- Python string datatype is a sequence made up of one or more individual characters where characters are letters, digits, whitespaces or any other symbol.
- Python treats strings as contiguous series of characters delimited by single, double or triple quotes.
- Python has built in string class “str”.
- We can declare a string in several ways
- Eg: name=“India”
 Country= name
 graduate=‘N’
 nationality=str(“Indian”)

Introduction

Indexing:

- Individual characters in a string are accessed using subscript [] operator.
- The expression in brackets is called index.
- The index specifies a member of an ordered set and here it specifies the character we want to access from given set of characters in a string.
- The index of first character is 0 and last is $n-1$ where n is number of characters in a string.
- If you try to exceed bounds that is below 0 and above $n-1$ the error occurs.

Traversing

- A string traversing is done by accessing characters from one index to another.

```
main.py
1 str = "Hello welcome to the world of Python"
2 i = 2
3 print(str[i]) # index as an integer
4 print(str[i*3+1]) #index use as an expression that evaluates to an integer
```

```
Output
l
e

=== Code Execution Successful ===
```

Concatenating, Appending and Multiplying String

- The word concatenating means to join together.
- In python concatenation of two strings can be done using “+” operator.

main.py

```
1 str1 = "Hello"  
2 str2 = "World"  
3 str3 = str1 + " " + str2  
4 print("The concatenated string is :", str3)
```

Output

```
The concatenated string is : Hello World
```

```
=== Code Execution Successful ===
```

Appending

- If you want to add one string at the end of another string then we will use “+=” .

```
1 message = "Hello, "  
2 name = input("\n Enter your name : ")  
3 message += name  
4 message += " Welcome to Dayananda Sagar University OOPs Class. "  
5 print(message)  
6
```

Output

```
Enter your name : Anurag  
Hello, Anurag Welcome to Dayananda Sagar University OOPs Class.  
=== Code Execution Successful ===
```

Multiplying String

- “*” operator can be used to repeat a string n number of times.

main.py

```
1 message = "Hello,"  
2 print(message * 3)
```

Output

```
Hello,Hello,Hello,
```

```
=== Code Execution Successful ===
```

- The str() function is used to convert values of any other type into string type.
- This helps the programmer to concatenate a string with any other datatype data.

main.py

```
1 message = "Hello"  
2 var = 7  
3 message2 = message + str(var)  
4 print(message2)
```

Output

```
Hello7
```

```
=== Code Execution Successful ===
```

Contd...

- Print statement prints one or more literals or values in a newline.
- If you don't want in newline then we can use "end=' '".

main.py

```
1 print("Hello", end = ' ')\n2 print("World")
```

Output

Hello World

=== Code Execution Successful ===

Immutable

- Python strings are immutable which means that once created they cannot be changed.
- Whenever you try to modify an existing string variable a new string is created.
- Every object in python is stored in memory.
- We can check whether two object is referring to same object or not by using Id().
- Id() returns memory address of that object.

main.py

```
1 message1 = "Hello"
2 print("Message1 is :", message1)
3 print("ID of message1 is : ", id(message1))
4
5 message2 = "World"
6 print("Message2 is : ", message2)
7 print("ID of message2 is : ", id(message2))
8
9 message1 += message2
10 print("Message1 after concatenation is :", message1)
11 print("ID of message1 is : ", id(message1))
12
```

Output

```
Message1 is : Hello
ID of message1 is : 136958919957728
Message2 is : World
ID of message2 is : 136958919958160
Message1 after concatenation is : HelloWorld
ID of message1 is : 136958919974448

=== Code Execution Successful ===
```

String Formatting Operator

- The ‘%’ operator takes a format string on left and the corresponding values in a tuple on the right.
- It allows users to construct string replacing part of string with the data stored in variables.
- The syntax for string formatting operation is
- The statement begins with a format string

```
"<FORMAT>" % (<VALUES>)
```

Consisting of a sequence of character and Conversion specifications.

Common Format Codes

%s → String

%d → Integer (decimal)

%f → Floating-point

%.nf → Floating-point with n decimal places

```
name = "Aarish"  
age = 8  
print("Name = %s and Age = %d" %(name, age))  
print("Name = %s and Age = %d" %("Anika", 6))
```

OUTPUT

```
Name = Aarish and Age = 8  
Name = Anika and Age = 6
```

Contd..

Floating-point Formatting

main.py

```
1 pi = 3.14159
2 print("Value of pi up to 2 decimal places: %.2f" % pi)
3
```

Output

```
Value of pi up to 2 decimal places: 3.14
=== Code Execution Successful ===
```

Multiple Values

main.py

```
1 item = "Book"
2 price = 499.99
3
4 print("The %s costs Rs %.2f" % (item, price))
5
```

Output

```
The Book costs Rs 499.99
=== Code Execution Successful ===
```

Built in string Methods and Function

- String is an example of python objects.
- Python supports many built in methods to manipulate strings.
- A method is just like function.
- Method is invoked or called on an object.

Function	Syntax	Usage	Example
capitalize()	string.capitalize()	Converts the first character to upper case	txt = "hello, and welcome to my world." x = txt.capitalize() print (x)
casefold()	string.casefold()	Converts string into lower case	txt = "Hello, And Welcome To My World!" x = txt.casefold() print(x)
center()	string.center(length, character)	Returns a centered string	txt = "banana" x = txt.center(20) print(x)
count()	string.count(value, start, end)	Returns the number of times a specified value occurs in a string	txt = "I love apples, apple are my favorite fruit" x = txt.count("apple") print(x)
endswith()	string.endswith(value, start, end)	Returns true if the string ends with the specified value	txt = "Hello, welcome to my world." x = txt.endswith(".") print(x)

Function	Syntax	Usage	Example
find()	s.find(substring, start, end))	Searches the string for a specified value and returns the position of where it was found	txt = "Hello, welcome to my world." x = txt.find("welcome") print(x)
isascii()	string.isascii()	Returns True if all characters in the string are ascii characters	txt = "Company123" x = txt.isascii() print(x)
isnumeric()	string.isnumeric()	Returns True if all characters in the string are numeric	txt = "565543" x = txt.isnumeric() print(x)
isupper()	string.isupper()	Returns True if all characters in the string are upper case	txt = "THIS IS NOW!" x = txt.isupper() print(x)
split()	string.split(separator, maxsplit)	Splits the string at the specified separator, and returns a list	txt = "welcome to the jungle" x = txt.split() print(x)

Function	Usage	Example
title()	Returns string in title case.	<pre>str = "The world is beautiful" print(str.title())</pre> OUTPUT The World Is Beautiful
swapcase()	Toggles the case of every character (uppercase character becomes lowercase and vice versa).	<pre>str = "The World Is Beautiful" print(str.swapcase())</pre> OUTPUT the world is beautiful
split(delim)	Returns a list of substrings separated by the specified delimiter. If no delimiter is specified then by default it splits strings on all whitespace characters.	<pre>str = "abc,def, ghi,jkl" print(str.split(','))</pre> OUTPUT ['abc', 'def', ' ghi', 'jkl']
join(list)	It is just the opposite of split. The function joins a list of strings using the delimiter with which the function is invoked.	<pre>print('-'.join(['abc', 'def', 'ghi', 'jkl']))</pre> OUTPUT abc-def-ghi-jkl
isidentifier()	Returns True if the string is a valid identifier.	<pre>str = "Hello" print(str.isidentifier())</pre> OUTPUT True
enumerate(str)	Returns an enumerate object that lists the index and value of all the characters in the string as pairs.	<pre>str = "Hello world" print(list(enumerate(str)))</pre> OUTPUT [(0, 'H'), (1, 'e'), (2, 'l'), (3, 'l'), (4, 'o'), (5, ' '), (6, 'w'), (7, 'o'), (8, 'r'), (9, 'l'), (10, 'd')]

Format()

- It is used for formatting strings
- Format strings have curly braces {} as placeholders or replacement fields which gets replaced.
- We can use positional arguments or keyword arguments to specify the order of fields that have to be replaced.

Multiplying String

- It returns a list of lines in the string
- It uses the newline characters such as \r and \n to split lines.
- Linebreaks are not included unless keepends are True.
- Syntax :
- str.splitlines(Keepends)

```
print('Sun and \n\n Stars, Planets \r and Moon\r\n'.splitlines())  
print('Sun and \n\n Stars, Planets \r and Moon\r\n'.splitlines(True))
```

OUTPUT

```
['Sun and ', '', ' Stars, Planets ', ' and Moon']  
['Sun and \n', '\n', ' Stars, Planets \r', ' and Moon\r\n']
```

Slice Operation

- A substring of a string is called a slice.
- Slice operation is used to refer to the sub-parts of sequences and strings.
- We can take a subset of a string from the original string by using [] operator also known as slicing operator.
- Syntax of slice operation is : `s[start: end]` where start indicates beginning index of the substring and end-1 is the index of the last character.
- Omitting either start or end index by default takes start or end of the string. Omitting both means the entire string.
- Python gives flexibility to access the string either from the first or the last character.
- If we are accessing the string from first character then we can use zero as index value, when doing it backward index starts with -1.

```
str = "PYTHON"
print("str[1:5] = ", str[1:5])
# to but not including index 5
print("str[:6] = ", str[:6])
print("str[1:] = ", str[1:])
print("str[:] = ", str[:])
print("str[1:20] = ", str[1:20])
# length of the string
```

OUTPUT

```
str[1:5] =  YTHO
str[:6] =  PYTHON
str[1:] =  YTHON
str[:] =  PYTHON
str[1:20] =  YTHON
```

```
str = "PYTHON"
print("str[-1] = ", str[-1])  # last character is accessed
print("str[-6] = ", str[-6])  # first character is accessed
```

```
print("str[-2:] = ", str[-2:]) # se
print("str[:-2] = ", str[:-2]) # al
character
print("str[-5:-2] = ", str[-5:-2])
accessed
```

OUTPUT

```
str[-1] =  N
str[-6] =  P
str[-2:] =  ON
str[:-2] =  PYTH
str[-5:-2] =  YTH
```

Specifying stride/stepsize while slicing the string

- In slice operation we have the option to specify stride as third argument, which refers to number of characters to move forward after the first character is retrieved from the string.
- The default value of the stride is 1.
- Even whitespace characters are also skipped because they are part of a string.

```
str = "Welcome to the world of Python"
print("str[::-3] = ", str[::-3])
```

OUTPUT

```
str[::-3] = WceohwloPh
```

```
str = "Welcome to the world of Python"
print("str[2:10] = ", str[2:10])    # c
print("str[2:10:1] = ", str[2:10:1])
print("str[2:10:2] = ", str[2:10:2])
print("str[2:13:4] = ", str[2:13:4])
```

OUTPUT

```
str[2:10] = lcome to
str[2:10:1] = lcome to
str[2:10:2] = loet
str[2:13:4] = le
```

Contd..

- Negative value can be also used as stride value.
- With -3, we have skipped every third letter of the string str.

```
str = "Welcome to the world of Python"  
print("str[::-1] = ", str[::-1])
```

OUTPUT

```
str[::-1] = nohtyP fo dlrow eht ot emoclew
```

```
str = "Welcome to the world of Python"  
print("str[::-3]", str[::-3])
```

OUTPUT

```
str[::-3] = nt r ttml
```

ord() and chr() functions

- The ord() function returns the ASCII code of the character.
- chr() function returns character represented by the ASCII number.

<pre>ch = 'R' print(ord(ch))</pre>	<pre>print(chr(82))</pre>	<pre>print(chr(112))</pre>	<pre>print(ord('p'))</pre>
OUTPUT	OUTPUT	OUTPUT	OUTPUT
82	R	p	112

In() and not in() operator

Membership Operator

- In() and not in() are used in string to determine whether a string is present in another string.
- They are also known as **membership operator**.

```
str1 = "Welcome to the world of Python  
!!!"  
str2 = "the"  
if str2 in str1:  
    print("Found")  
else:  
    print("Not Found")
```

OUTPUT

Found

```
str1 = "This is a very good book"  
str2 = "best"  
if str2 not in str1:  
    print("The book is very good but it  
may not be the best one.")  
else:  
    print ("It is the best book.")
```

OUTPUT

The book is very good but it may not be
the best one.

Contd...

- In() and not in() can be used to check whether a character is present in a word.

```
>>> 'u' in "stars"  
False
```

```
>>> 'v' not in "success"  
True
```

```
>>> 'vi' in "victory"  
True
```

```
>>> "world" in "world"  
True
```

```
>>> 'a' in 'a'  
True
```

Comparing Strings

Python allows you to compare the strings using relational or comparison operators such as `<`, `>`, `<=`, `>=` etc.

Operator	Description	Example
<code>==</code>	If two strings are equal, it returns True.	<pre>>>> "AbC" == "AbC" True</pre>
<code>!=</code> or <code><></code>	If two strings are not equal, it returns True.	<pre>>>> "AbC" != "Abc" True >>> "abc" <> "ABC" True</pre>
<code>></code>	If the first string is greater than the second, it returns True.	<pre>>>> "abc" > "Abc" True</pre>
<code><</code>	If the second string is greater than the first, it returns True.	<pre>>>> "abC" < "abc" True</pre>
<code>>=</code>	If the first string is greater than or equal to the second, it returns True.	<pre>>>> "aBC" >= "ABC" True</pre>
<code><=</code>	If the second string is greater than or equal to the first, it returns True.	<pre>>>> "ABc" <= "ABc" True</pre>

Contd...

- These operators compares the string by using lexicographical order i.e using ASCII value of the character.
- The ASCII value of A-Z is 65-90 and ASCII code of a-z 97-122.

```
>>> "TED" == "ted"
```

```
False
```

```
>>> "True" >= "False"
```

```
True
```

```
>>> "talk" > "talks"
```

```
False
```

```
>>> "like" != "likes"
```

```
True
```

```
>>> "Main" < "main"
```

```
True
```

```
>>> "tend" <= "tent"
```

```
True
```

Iterating String

- String is a sequence type(sequence of characters).
- We can iterate through strings using for loop or while loop.

```
str = "Welcome to Python"  
for i in str:  
    print(i, end=' ')
```

OUTPUT

W e l c o m e t o P y t h o n

```
message = " Welcome to Python "  
index = 0  
while index < len(message):  
    letter = message[index]  
    print(letter, end=' ')  
    index += 1
```

OUTPUT

W e l c o m e t o P y t h o n